



# Модуль 1. Фундамент Python: то, без чего асинхронность — магия

## Введение в модуль

Прежде чем мы откроем для себя `async` и `await`, нужно понять три механизма Python, которые лежат в их основе:

1. **Итераторы и генераторы** — потому что `async for` и `async with` — это прямые потомки обычных `for` и `with`, построенные на тех же протоколах.
2. **Менеджеры контекста** — потому что управление жизненным циклом ресурсов (соединений с БД, сессий ML-моделей) невозможно без понимания `__enter__` / `__exit__` и их асинхронных двойников.
3. **Система типов** — потому что FastAPI и Pydantic V2 построены на идее, что типы — это не просто подсказки, а исполняемые контракты.

Мы пойдём от первых принципов: от того, как Python выполняет цикл `for`, до теории категорий, стоящей за вариантностью типов.

## 1.1. Итераторы и генераторы

### 1.1.1. Что происходит, когда вы пишете `for x in collection`?

В Python цикл `for` — это не конструкция языка низкого уровня (как в C), а **синтаксический сахар** над протоколом итератора. Интерпретатор не знает заранее, как устроена ваша коллекция. Он знает только, что у неё есть метод `__iter__`.

Разберём по шагам. Когда вы пишете:

```
In [ ]: for element in [10, 20, 30]:  
        print(element)
```

Python под капотом выполняет примерно следующее:

```
In [ ]: _iterator = iter([10, 20, 30]) # вызывает [10, 20, 30].__iter__()
while True:
    try:
        element = next(_iterator) # вызывает _iterator.__next__()
        print(element)
    except StopIteration:
        break
```

Это и есть **протокол итератора**. Он состоит из двух методов:

| Метод                       | Назначение                                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>__iter__(self)</code> | Возвращает объект, у которого есть метод <code>__next__</code> . Обычно возвращает <code>self</code> .               |
| <code>__next__(self)</code> | Возвращает следующий элемент последовательности. Если элементы закончились, выбрасывает <code>StopIteration</code> . |

**Важное уточнение:** в Python различают **итерируемый объект** (iterable) и **итератор** (iterator).

- **Итерируемый объект** — тот, у кого есть `__iter__`. Примеры: `list`, `str`, `dict`, `set`, `range`.
- **Итератор** — тот, у кого есть и `__iter__`, и `__next__`. Итератор — это *машина состояний*, которая помнит, на каком элементе остановилась.

Почему `list` не является итератором? Потому что вы можете пройти по одному списку несколько раз:

```
In [ ]: data = [1, 2, 3]
for x in data: print(x)
for x in data: print(x) # работает снова
```

Каждый вызов `iter(data)` создаёт **новый** итератор. Сам список при этом не изменяется.

## 1.1.2. Пишем свой итератор с нуля

Чтобы понять механику, напишем итератор, который генерирует квадраты чисел от `start` до `end`:

```
In [1]: class SquareIterator:
        def __init__(self, start: int, end: int):
            self.current = start
            self.end = end
```

```

def __iter__(self):
    # Итератор должен возвращать self из __iter__,
    # чтобы работал вложенный for или распаковка
    return self

def __next__(self):
    if self.current > self.end:
        raise StopIteration
    value = self.current ** 2
    self.current += 1
    return value

# Использование
sq = SquareIterator(1, 3)
for x in sq:
    print(x)  # 1, 4, 9

```

```

1
4
9

```

```

In [3]: # повторный вызов ничего не даст, т.к. под капотом сразу вылетает StopIteration
for x in sq:
    print(x)

```

Здесь `SquareIterator` — это **объект с состоянием**. У него есть поле `self.current`, которое меняется при каждом вызове `__next__`.

Итератор **исчерпаем**: после одного прохода он навсегда останется в состоянии `current > end`, и второй `for` по тому же объекту ничего не выдаст.

### 1.1.3. Математическая подоплека: последовательности как отображения

В математике **последовательность** — это функция, сопоставляющая каждому натуральному числу  $n$  элемент  $a_n$ :

$\$a: \mathbb{N} \rightarrow X$

где  $X$  — некоторое множество (тип элементов).

Итератор в Python — это не вся последовательность сразу, а **ленивое воплощение** этого отображения. Он не хранит все  $a_1, a_2, \dots, a_n$  в памяти. Он хранит только текущее состояние (текущее  $n$ ) и правило перехода к следующему элементу.

Это позволяет работать с **бесконечными последовательностями**:

```

In [4]: class InfiniteCounter:

```

```

def __init__(self, start=0):
    self.n = start

def __iter__(self):
    return self

def __next__(self):
    value = self.n
    self.n += 1
    return value

counter = InfiniteCounter()
for i in counter:
    if i > 5:
        break
    print(i)

```

```

0
1
2
3
4
5

```

Математически это соответствует последовательности  $a_n = n$  для  $n \in \mathbb{N}_0$ . В памяти при этом хранится только одно число `self.n`.

### 1.1.4. Генераторы: итераторы, которые пишутся как функции

Написание класса с `__iter__` и `__next__` — многословно. Python предлагает сокращение: **генераторную функцию** — функцию, содержащую ключевое слово `yield`.

```

In [5]: def squares(start, end):
        current = start
        while current <= end:
            yield current ** 2
            current += 1

        # Использование
        for x in squares(1, 3):
            print(x) # 1, 4, 9

```

```

1
4
9

```

```

In [8]: for x in squares(1, 3):
        print(x) # 1, 4, 9

```

```
1
4
9
```

Что происходит здесь?

1. Вызов `squares(1, 3)` **не выполняет тело функции**. Он возвращает объект-генератор.
2. Когда `for` вызывает `next()` на этом генераторе, выполнение начинается с первой строки и идёт до `yield`.
3. `yield current ** 2` возвращает значение, но **сохраняет всё локальное состояние** функции: значения переменных `current`, `start`, `end`, и точку возврата.
4. При следующем `next()` выполнение продолжается сразу после `yield` — инкрементируется `current`, проверяется условие цикла, и снова встречается `yield`.

Генератор — это **сопрограмма** (coroutine) в узком смысле: функция, которая может приостанавливать своё выполнение и возобновлять его позже, сохраняя локальное состояние.

### 1.1.5. Состояние генератора: взгляд изнутри

Можно исследовать объект-генератор:

```
In [9]: gen = squares(1, 3)
print(gen) # <generator object squares at 0x...>

print(next(gen)) # 1
print(next(gen)) # 4
print(next(gen)) # 9
print(next(gen)) # StopIteration
```

```
<generator object squares at 0x00000283847D7ED0>
```

```
1
4
9
```

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[9], line 7
      5 print(next(gen)) # 4
      6 print(next(gen)) # 9
----> 7 print(next(gen)) # StopIteration

StopIteration:
```

Генератор имеет методы:

- `gen.__next__()` — получить следующее значение (или `StopIteration`).
- `gen.send(value)` — отправить значение *внутри* генератора (используется в продвинутых сопрограмах).
- `gen.throw(type, ...)` — выбросить исключение внутри генератора.
- `gen.close()` — принудительно завершить генератор.

Когда генератор завершается (выпадает из функции или доходит до `return`), он выбрасывает `StopIteration`. Именно поэтому цикл `for` знает, когда остановиться.

### 1.1.6. Генераторные выражения

Есть ещё более компактная форма — **генераторное выражение** (generator expression):

```
In [10... squares_gen = (x**2 for x in range(1, 4))
```

Это ленивый аналог list comprehension `[x**2 for x in range(1, 4)]`. Разница критична:

```
In [22... import sys

# List comprehension — создаёт весь список в памяти
squares_list = [x**2 for x in range(10_000_000)] # ~100 МБ памяти
print(f"list comprehension: {sys.getsizeof(squares_list)/1024/1024:.2f} M")

# Generator expression — создаёт объект, выдающий элементы по одному
squares_gen = (x**2 for x in range(10_000_000)) # ~200 байт памяти
print(f"generator expression: {sys.getsizeof(squares_gen)} B")
```

```
list comprehension: 84.97 MB
generator expression: 208 B
```

Генераторное выражение — это анонимный генератор. Его можно использовать для конвейерной обработки данных:

```
In [ ]: lines = (line.strip() for line in open("data.txt"))
numbers = (int(x) for x in lines if x.isdigit())
evens = (x for x in numbers if x % 2 == 0)
result = sum(evens) # Ни один промежуточный список не создаётся!
```

### 1.1.7. Вложенные генераторы: `yield from`

Python 3.3 ввёл оператор `yield from`, который позволяет делегировать

генерацию другому итератору:

```
In [23... def sub_generator():  
    yield 1  
    yield 2  
  
def main_generator():  
    yield "start"  
    yield from sub_generator()  
    yield "end"  
  
for x in main_generator():  
    print(x) # start, 1, 2, end
```

```
start  
1  
2  
end
```

`yield from` не просто цикл. Он устанавливает **двунаправленный канал**: значения, отправляемые через `.send()`, и исключения, брошенные через `.throw()`, прозрачно проходят через делегирующий генератор к подчинённому.

Это важно, потому что именно `yield from` лежит в основе `await` в `asyncio`. `await` — это синтаксический сахар над `yield from` для корутин.

### 1.1.8. Математическая подоплека: ленивые списки и теория типов

В теории типов и функциональном программировании существует понятие **ленивого списка** (lazy list) или **потока** (stream). Это структура данных, определённая рекурсивно:

$$\text{Stream}(A) = \text{Nil} \mid \text{Cons}(A, \text{Stream}(A))$$

где голова ( `head` ) вычисляется сразу, а хвост ( `tail` ) — только при обращении.

Генераторы в Python — императивная реализация этой идеи. Вместо рекурсивной структуры данных у нас есть объект с внутренним состоянием, но семантика та же: бесконечные последовательности, обрабатываемые поэлементно, без полной материализации в памяти.

## 1.2. Менеджеры контекста

## 1.2.1. Проблема: ресурсы нужно освобождать

Рассмотрим типичный сценарий работы с файлом:

```
In [ ]: f = open("data.txt", "r")
data = f.read()
# Если здесь произойдёт исключение — файл останется открытым!
f.close()
```

Если между `open` и `close` выпадет исключение (например, `data = f.read()` вызовет `MemoryError`), `f.close()` никогда не выполнится. Файловый дескриптор останется занят, что на Unix-системах может привести к исчерпанию лимита `ulimit -n`.

Классическое решение — `try...finally`:

```
In [ ]: f = open("data.txt", "r")
try:
    data = f.read()
finally:
    f.close() # выполнится всегда, даже при исключении
```

Но это многословно. А если ресурсов несколько?

```
In [ ]: f1 = open("a.txt")
try:
    f2 = open("b.txt")
    try:
        # работа с f1 и f2
        pass
    finally:
        f2.close()
finally:
    f1.close()
```

Код превращается в «лестницу судьбы». Менеджеры контекста решают эту проблему.

## 1.2.2. Протокол менеджера контекста

Менеджер контекста — это объект, реализующий два метода:

| Метод                        | Когда вызывается                   | Что делает                                                              |
|------------------------------|------------------------------------|-------------------------------------------------------------------------|
| <code>__enter__(self)</code> | При входе в блок <code>with</code> | Инициализация ресурса, возвращает объект, привязанный к <code>as</code> |



| Метод                                                  | Когда вызывается                                                     | Что делает                                             |
|--------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------|
| <code>__exit__(self, exc_type, exc_val, exc_tb)</code> | При выходе из блока <code>with</code> (нормальном или по исключению) | Освобождение ресурса, возможно — подавление исключения |

Пример — собственный менеджер для таймера:

```
In [25... import time
from typing import Optional

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self # возвращаем self, чтобы можно было написать "as t"

    def __exit__(
        self,
        exc_type: Optional[type],
        exc_val: Optional[BaseException],
        exc_tb: Optional[object]
    ) -> Optional[bool]:
        self.end = time.perf_counter()
        self.elapsed = self.end - self.start
        print(f"Прошло {self.elapsed:.4f} секунд")
        # Возвращаем None (или False) — исключение не подавляем
        return None

# Использование
with Timer() as t:
    time.sleep(0.2)
```

Прошло 0.2007 секунд

### 1.2.3. Как работает `with` под капотом

Конструкция:

```
In [ ]: with EXPR as VAR:
        BLOCK
```

Python транслирует примерно в:

```
In [ ]: manager = (EXPR)
enter = type(manager).__enter__
exit = type(manager).__exit__
value = enter(manager)
hit_except = False

try:
    VAR = value
```

```

BLOCK
except:
    hit_except = True
    if not exit(manager, *sys.exc_info()):
        raise # если __exit__ вернул False/None – пробрасываем исключение
finally:
    if not hit_except:
        exit(manager, None, None, None)

```

Ключевой момент: `__exit__` вызывается **всегда**, независимо от того, было ли исключение в блоке.

## 1.2.4. Подавление исключений в `__exit__`

Метод `__exit__` может возвращать `True`, чтобы **подавить** исключение:

```

In [26... class SuppressZeroDivision:
    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is ZeroDivisionError:
            print("Деление на ноль подавлено")
            return True # <- исключение не всплывает дальше
            return False # <- другие исключения пробрасываются

with SuppressZeroDivision():
    print(1 / 0) # не упадёт, выведет "Деление на ноль подавлено"

```

Деление на ноль подавлено

Это используется в стандартной библиотеке `contextlib.suppress`:

```

In [ ]: from contextlib import suppress

with suppress(FileNotFoundError):
    os.remove("temp.txt") # если файла нет – не упадём

```

## 1.2.5. `contextlib.contextmanager`: менеджеры через генераторы

Писать класс ради пары методов — утомительно. Декоратор `@contextmanager` позволяет создавать менеджеры контекста из **генераторных функций**:

```

In [27... from contextlib import contextmanager

```

```
@contextmanager
def managed_resource(name):
    print(f"Ресурс {name}: инициализация")
    resource = {"name": name, "active": True}
    try:
        yield resource # <- здесь происходит вход в блок with
    finally:
        resource["active"] = False
        print(f"Ресурс {name}: освобождение")

# Использование
with managed_resource("database") as db:
    print(f"Работаем с {db}")
# После выхода из with выполняется finally
```

```
Ресурс database: инициализация
Работаем с {'name': 'database', 'active': True}
Ресурс database: освобождение
```

Как это работает?

1. Функция `managed_resource` возвращает генератор.
2. `__enter__` вызывает `next()` на генераторе, выполнение идёт до `yield` и возвращает выданное значение.
3. `__exit__` вызывает `next()` снова (или `close()` при исключении), выполнение продолжается после `yield` — в блоке `finally`.

Это красиво, потому что повторно используется механизм генераторов (сохранение состояния) для управления ресурсами.

## 1.2.6. Асинхронные менеджеры контекста

В асинхронном коде нельзя просто вызвать `yield` — нужны `await`. Поэтому существуют асинхронные аналоги:

```
In [ ]: from contextlib import asynccontextmanager

@asynccontextmanager
async def async_managed_resource(name):
    print(f"Async ресурс {name}: подключение")
    conn = await create_connection(name) # асинхронная инициализация
    try:
        yield conn
    finally:
        await conn.close() # асинхронное освобождение
```

Протокол асинхронного менеджера:

| Метод                  | Асинхронный аналог                                      |
|------------------------|---------------------------------------------------------|
| <code>__enter__</code> | <code>__aenter__(self)</code>                           |
| <code>__exit__</code>  | <code>__aexit__(self, exc_type, exc_val, exc_tb)</code> |

Использование:

```
In [ ]: async with async_managed_resource("postgres") as conn:
        result = await conn.fetch("SELECT * FROM users")
```

Это прямой предшественник паттерна `lifespan` в FastAPI, где `startup` — это `__aenter__`, а `shutdown` — `__aexit__`.

## 1.2.7. Математическая подоплека: инварианты и гарантии

В теории программирования менеджер контекста реализует паттерн **RAII** (Resource Acquisition Is Initialization), пришедший из C++.

Формально можно сказать, что `with` создаёт **область видимости**, внутри которой выполняется инвариант: «ресурс открыт». При входе в область инвариант устанавливается (`__enter__`), при выходе — гарантированно снимается (`__exit__`), даже если внутри произошло исключение.

Это частный случай более общей концепции **алгебраических эффектов** в теории языков программирования, где `with` — это обработчик эффекта «получение и освобождение ресурса».

## 1.3. Типизация в Python

### 1.3.1. От динамической типизации к аннотациям

Python — язык с **динамической типизацией**. Переменная не имеет типа; тип есть у значения:

```
In [ ]: x = 42          # x ссылается на int
        x = "hello"    # теперь x ссылается на str
```

Это гибко, но при масштабировании проекта приводит к ошибкам: функция ожидает `list`, а получает `None`; метод ожидает `User`, а

получает `dict`.

С PEP 484 (Python 3.5+) появились **аннотации типов** — подсказки, которые не влияют на выполнение, но позволяют статическим анализаторам (`mypy`, `pyright`) проверять корректность.

```
In [ ]: def greet(name: str) -> str:
        return f"Hello, {name}"
```

Во время выполнения аннотации игнорируются. Но инструменты проверяют, что `greet(42)` — ошибка.

### 1.3.2. Базовые типы из модуля `typing`

```
In [ ]: from typing import List, Dict, Tuple, Optional, Union, Callable

def process(data: List[int]) -> Dict[str, int]:
    return {"sum": sum(data)}

def find(user_id: int) -> Optional[str]:
    # Optional[str] == Union[str, None]
    if user_id > 0:
        return "Alice"
    return None

def apply(func: Callable[[int], bool], items: List[int]) -> List[int]:
    return [x for x in items if func(x)]
```

В Python 3.9+ встроенные коллекции (`list`, `dict`, `tuple`) поддерживают индексацию, и `typing.List` стал необязательным:

```
In [ ]: def process(data: list[int]) -> dict[str, int]:
        ...
```

### 1.3.3. Обобщённые типы: `Generic` и `TypeVar`

Часто нужно написать функцию или класс, который работает с **любым** типом, но сохраняет конкретный тип на протяжении использования. Например, стек, который хранит элементы одного типа:

```
In [ ]: from typing import TypeVar, Generic, List

T = TypeVar('T') # переменная типа, "placeholder"

class Stack(Generic[T]):
    def __init__(self) -> None:
```

```

    self._items: List[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

# Использование
int_stack: Stack[int] = Stack()
int_stack.push(42)
# int_stack.push("oops") # туру выдаст ошибку!

str_stack: Stack[str] = Stack()
str_stack.push("hello")

```

`TypeVar('T')` создаёт **переменную типа**. `Generic[T]` объявляет, что класс параметризован типом `T`. Это называется **параметрическим полиморфизмом** — функция/класс работает одинаково для любого типа, но конкретный экземпляр фиксирует тип.

### 1.3.4. Вариантность типов: ковариантность, контравариантность, инвариантность

Это — самый глубокий теоретический раздел. Он критичен для понимания, почему `List[Cat]` не является `List[Animal]`, но `Callable[[Animal], None]` является подтипом `Callable[[Cat], None]`.

#### Подтипы

Если `Cat` — подкласс `Animal`, пишем:

```

$$\text{Cat} <: \text{Animal}

```

(читается: «Cat является подтипом Animal»).

Вопрос: как относятся друг к другу `Container[Cat]` и `Container[Animal]`?

#### Ковариантность (Covariance)

Контейнер **ковариантен**, если сохраняет направление иерархии:

```

$$\text{Cat} <: \text{Animal} \implies \text{Container}[\text{Cat}] <: \text{Container}[\text{Animal}]

```

**Пример:** `Iterable[T]` ковариантен.

```
In [ ]: from typing import Iterable

def count_pets(pets: Iterable[Animal]) -> int:
    return sum(1 for _ in pets)

cats: list[Cat] = [Cat(), Cat()]
count_pets(cats) # OK: Iterable[Cat] <: Iterable[Animal]
```

Почему это безопасно? Потому что `Iterable` только **выдаёт** элементы. Мы читаем `Animal` из контейнера, и если там на самом деле `Cat` — это нормально, ведь `Cat` — это `Animal`.

Обозначение в Python: `T_co = TypeVar('T_co', covariant=True)`

## Контравариантность (Contravariance)

Контейнер **контравариантен**, если **обращает** направление иерархии:

$\text{Cat} <: \text{Animal} \implies \text{Container}[\text{Animal}] <: \text{Container}[\text{Cat}]$

**Пример:** `Callable[[T], None]` контравариантен по аргументу.

```
In [ ]: from typing import Callable

def feed_animal(a: Animal) -> None:
    print(f"Feeding {a.name}")

def process_cat(handler: Callable[[Cat], None]) -> None:
    cat = Cat()
    handler(cat)

process_cat(feed_animal) # OK!
```

Почему это безопасно? `process_cat` ожидает функцию, которая может принять `Cat`. `feed_animal` принимает `Animal`. Поскольку `Cat` — это `Animal`, `feed_animal` справится с `Cat`. Но наоборот не работало бы: функция, принимающая только `Cat`, не сможет обработать произвольное `Animal`.

Обозначение: `T_contra = TypeVar('T_contra', contravariant=True)`

## Инвариантность (Invariance)

Контейнер **инвариантен**, если иерархия не сохраняется ни в каком направлении:

$\text{Container}[\text{Cat}] \not<: \text{Container}[\text{Animal}] \quad \text{и} \quad \text{Container}[\text{Animal}] \not<: \text{Container}[\text{Cat}]$

$\text{Container}[\text{Animal}] \not<: \text{Container}[\text{Cat}]$

**Пример:** `List[T]` инвариантен.

```
In [ ]: def add_dog(animals: list[Animal]) -> None:
        animals.append(Dog())

cats: list[Cat] = [Cat(), Cat()]
add_dog(cats)  # ОПАСНО! туру запретит это.
```

Если бы `list[Cat] <: list[Animal]` было разрешено, функция `add_dog` добавила бы `Dog` в список кошек — нарушение типовой безопасности.

По умолчанию `TypeVar` создаёт инвариантные переменные.

### Сводная таблица

| Вариантность       | Направление | Пример в Python                                           | Интуиция               |
|--------------------|-------------|-----------------------------------------------------------|------------------------|
| Ковариантность     | Сохраняет   | <code>Iterable[+T]</code> , <code>Callable[[], +T]</code> | Только читаем          |
| Контравариантность | Обращает    | <code>Callable[[-T], None]</code>                         | Только пишем/принимаем |
| Инвариантность     | Ни одно     | <code>List[T]</code> , <code>Dict[K, V]</code>            | И читаем, и пишем      |

### 1.3.5. Математическая подоплека: теория категорий

Вариантность типов — это не прихоть разработчиков языка, а следствие **теории категорий**.

Рассмотрим функтор  $F$ : отображение из категории типов в категорию типов.

- **Ковариантный функтор** сохраняет направление морфизмов (функций): если есть  $f: A \rightarrow B$ , то есть  $F(f): F(A) \rightarrow F(B)$ . Пример: функтор  $F(X) = \text{Iterable}[X]$ .
- **Контравариантный функтор** обращает направление: если есть  $f: A \rightarrow B$ , то есть  $F(f): F(B) \rightarrow F(A)$ . Пример: функтор  $F(X) = \text{Callable}[[X], \text{None}]$ .

Инвариантные конструкторы типов (вроде `List`) не являются



функторами в чистом виде — они требуют, чтобы входной и выходной тип совпадали.

Это объясняет, почему в системах типов существуют именно эти три вариантности: они соответствуют функториальным свойствам конструкторов типов.

### 1.3.6. Структурная типизация: `Protocol`

Python использует **номинальную типизацию** по умолчанию: `class Dog(Animal)` означает, что `Dog` — подтип `Animal`, потому что мы явно указали наследование.

Но иногда важно не имя класса, а **структура**: какие методы и атрибуты есть у объекта. Это называется **утиная типизация** (duck typing) в динамическом Python, а в статическом анализе — **структурная типизация**.

`Protocol` (PEP 544) позволяет определять интерфейсы по структуре:

```
In [ ]: from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> None:
        ...

class Circle:
    def draw(self) -> None:
        print("Drawing circle")

class Rectangle:
    def draw(self) -> None:
        print("Drawing rectangle")

def render(items: list[Drawable]) -> None:
    for item in items:
        item.draw()

# Circle и Rectangle не наследуют Drawable,
# но тупо считает их совместимыми, потому что у них есть метод draw
render([Circle(), Rectangle()])
```

Это мощно для архитектуры: вы можете определять зависимости через интерфейсы (`Protocol`), не создавая жёстких иерархий наследования.

### 1.3.7. Pydantic: типы как runtime-контракты

Аннотации Python — это только подсказки для статического анализатора. Они не влияют на выполнение. Но FastAPI использует **Pydantic** — библиотеку, которая превращает аннотации в **runtime-валидаторы**.

```
In [ ]: from pydantic import BaseModel, Field

class User(BaseModel):
    id: int
    name: str = Field(min_length=1, max_length=100)
    email: str
    age: int = Field(ge=0, le=150)

# Pydantic проверит типы и ограничения во время выполнения!
user = User(id="42", name="Alice", email="alice@example.com", age=25)
# id="42" автоматически преобразуется в int 42
# Если передать age=-5 – будет ValidationError
```

Pydantic V2 переписал ядро валидации на **Rust** ( `pydantic-core` ). Это даёт:

- Валидацию JSON со скоростью, сравнимой с компилируемыми языками.
- Потребление памяти на порядок ниже, чем в Pydantic V1.
- Генерацию JSON Schema «из коробки».

Связь с типами: Pydantic берёт аннотации Python и строит из них **схему валидации**. `BaseModel` — это не просто `dataclass`, а **граничный контролёр** системы: всё, что входит извне (HTTP-запросы, JSON, формы), проходит через типизированную модель Pydantic.

## Итог модуля 1

| Концепция                                                                      | Зачем нужна                                      | Связь с асинхронностью                                                                      |
|--------------------------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Протокол итератора</b><br>( <code>__iter__</code> , <code>__next__</code> ) | Единый способ перебора любой коллекции           | <code>async for</code> использует <code>__aiter__</code> / <code>__anext__</code>           |
| <b>Генераторы</b><br>( <code>yield</code> )                                    | Ленивые последовательности, сохранение состояния | <code>async def</code> + <code>await</code> — это корутины, построенные на том же механизме |

| Концепция                                                                                | Зачем нужна                                  | Связь с асинхронностью                                             |
|------------------------------------------------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------|
| <b>Менеджеры контекста</b> ( <code>with</code> )                                         | Гарантированное освобождение ресурсов        | <code>async with</code> — для асинхронных ресурсов (соединения БД) |
| <b>Типизация</b> ( <code>Generic</code> , <code>TypeVar</code> , <code>Protocol</code> ) | Безопасность и документирование кода         | FastAPI строит валидацию и документацию из типов                   |
| <b>Вариантность</b> (ко/контра/инвариантность)                                           | Понимание, какие подстановки типов безопасны | Критична для проектирования generic-контейнеров и callback-ов      |

В **Модуле 2** мы перейдём к `asyncio`: увидим, как `yield` превращается в `await`, а генераторы — в корутины, управляемые event loop.